VLSI Design Course Project, Part II
SIMD-Reconfigurable 8-bit Adder

Bhargav, 2025102061
IIIT Hyderabad, Monsoon 2026

Contents

1 Task 2

2 Architecture 2

3 The carry-gating cell 3

4 Layout 5

5 Worst-case propagation delay in both modes 8

6 Delay penalty of the gating cell 9

7 Maximum frequency and power penalty 10

8 Energy-delay product 11

9 Summary and limits 12

A Netlists 12

B Reproducing the results 12

1 Task

Part II builds an 8-bit adder from two instances of the Part I 4-bit carry look-ahead adder, CLA0 for bits 3..0 and CLA1 for bits 7..4, with a control input SIMD_MODE:

- **SIMD_MODE = 0, 8-bit mode:** the carry-out of CLA0 propagates into the carry-in of CLA1.
- **SIMD_MODE = 1, dual 4-bit mode:** the inter-block carry is forced to 0, so CLA0 and CLA1 add two independent 4-bit numbers.

A carry-gating cell between the two blocks does the switching. Its cost is measured against a **baseline**: the same two adders with CLA0's carry-out wired straight to CLA1's carry-in.

Everything here uses the provided TSMC 180 nm model in ngspice at 1.8 V, and MAGIC with the provided SCN6M_DEEP technology file, $\lambda = 0.09\,\mu$m. Both 4-bit blocks are the final Part I adder with its final transistor sizes. Every output drives a $W_p/W_n = 20\lambda/10\lambda$ inverter, and every input is driven through a shaper and an x2 inverter, as in Part I.

2 Architecture

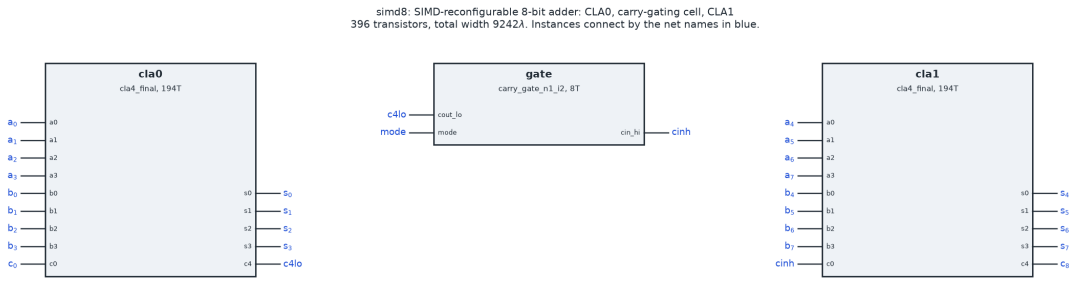


Figure 1: SIMD 8-bit adder: CLA0, the carry-gating cell and CLA1. c4lo is CLA0's carry-out, cinh is CLA1's carry-in.

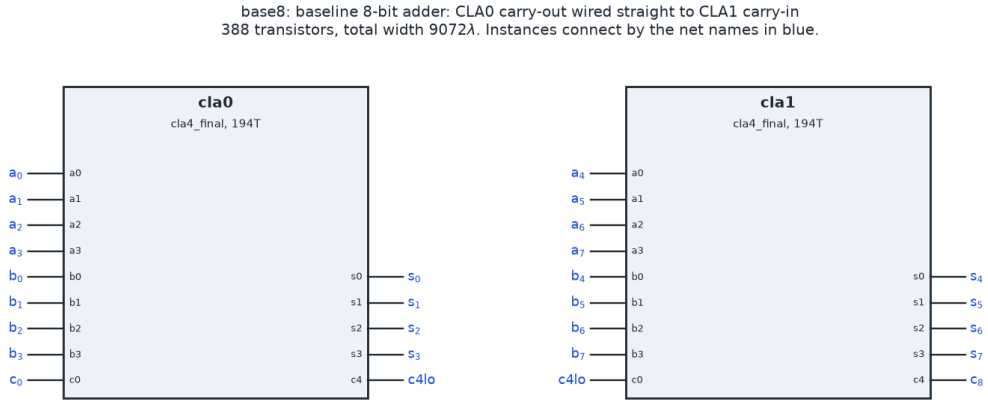


Figure 2: Baseline 8-bit adder: CLA0's carry-out c4lo goes straight to CLA1's carry-in.

The SIMD design also brings CLA0's carry-out to an output, c4lo, because in dual mode it is the carry of the lower 4-bit result. It drives the same $20\lambda/10\lambda$ load as the other outputs.

Functional check. Both netlists were evaluated at switch level on 3000 random inputs each, against the behavioural model: 0 errors.

3 The carry-gating cell

The carry into CLA1 must equal CLA0's carry-out in mode 0 and be 0 in mode 1:

$$c_{in,hi} = c_{out,lo} \cdot \overline{\text{SIMD_MODE}}$$

This is an AND gate. In static CMOS, the cell uses a NAND2 followed by an inverter on the carry path, and a unit inverter for $\overline{\text{SIMD_MODE}}$. SIMD_MODE is set before an add and held, so that inverter is off the timing path. Static CMOS was kept for the same reasons as in Part I: full swing and no static current.

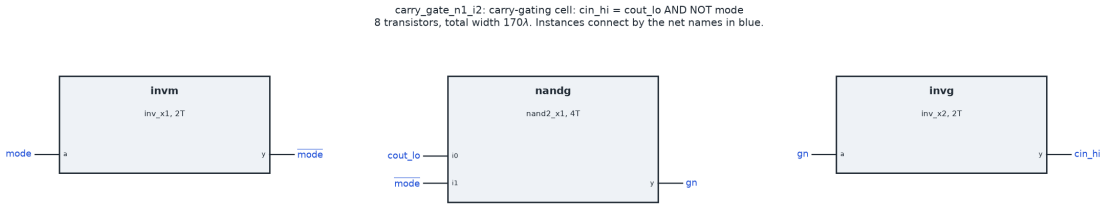


Figure 3: Carry-gating cell, 8 transistors.

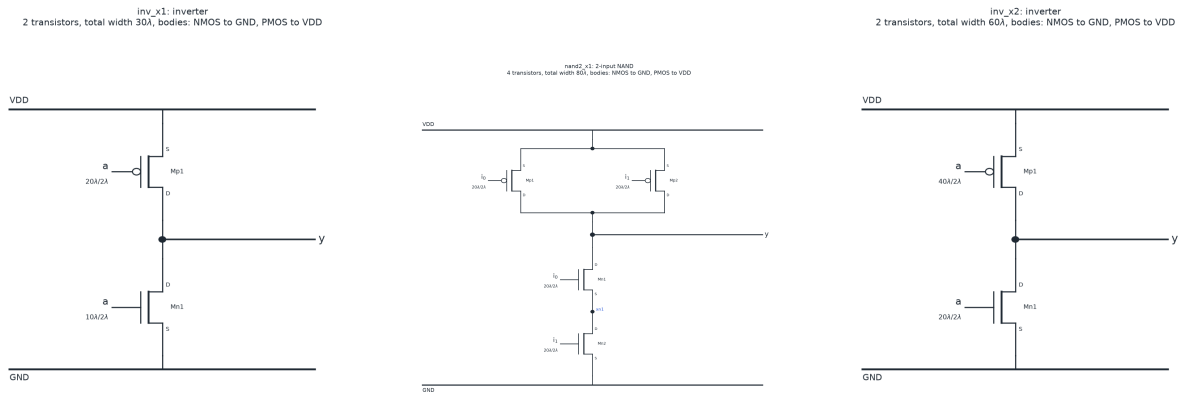


Figure 4: Transistor-level schematics: mode inverter, NAND2 on the carry path, output inverter.

Sizing. The NAND2 and the output inverter were swept on the schematic of the full SIMD adder in mode 0, measuring the worst delay from Cin to S7 on the vector A = FF, B = 00 (figure 5). The fastest was NAND2 x1 with an x2 output inverter, 696.59 ps. A larger NAND2 is slower: it adds load to CLA0's carry-out, which is on the critical path.

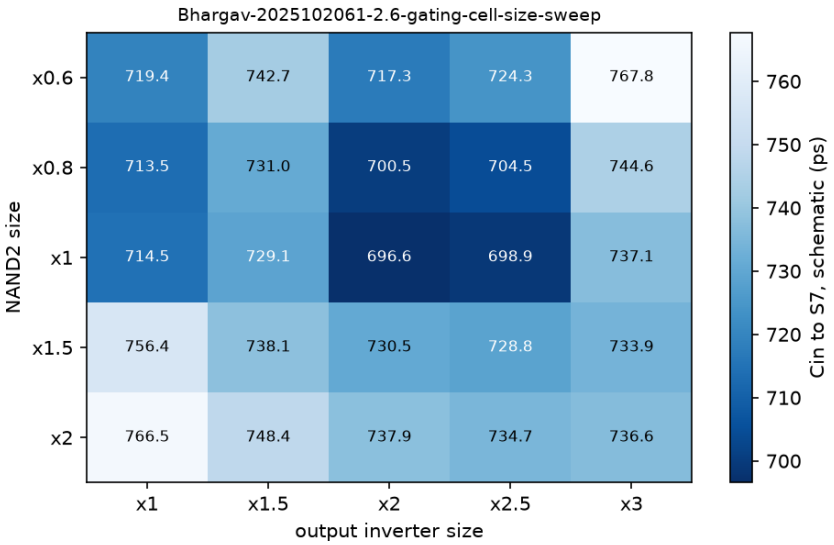


Figure 5: Gate sizing sweep: Cin to S7 in 8-bit mode, schematic.

Table 1: Carry-gating cell on its own, cout_lo to cin_hi with a 20/10λ load.

View	**Worst (ps)**	**Output rising (ps)**	**Output falling (ps)**
Schematic	104.27	89.12	104.27
Post-layout	110.46	95.20	110.46

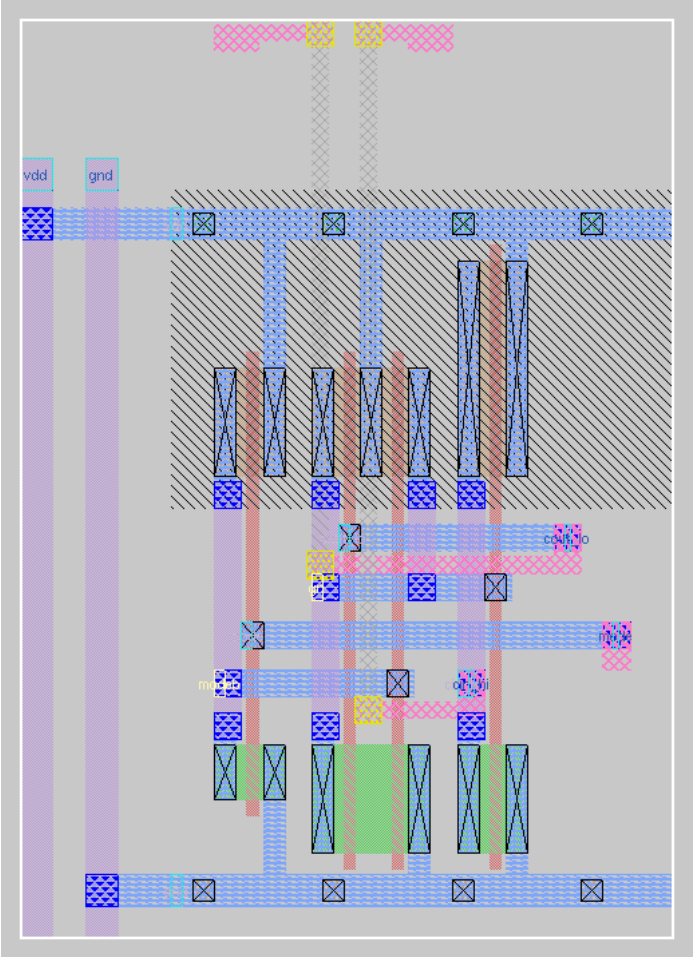


Figure 6: Gating cell in MAGIC, with its carry pads above it, as placed in the SIMD adder. The cell alone is 93 x 133 λ, DRC 0, LVS match: 8 merged devices, 8 nets.

4 Layout

The layout keeps the two adders as two placed copies of one cell, as the task describes.

1. **Adder cell.** The Part I adder is laid out again by the Part I flow, with two extra metal 3 pads above its top row for C4 and C0. They are routed inside the cell like any other pin of those nets, so the top level can land a via on plain metal 3. The cell is DRC 0, LVS match: 194 merged devices, 108 nets, 3083.1 μm^2.

2. **Gating cell.** Laid out by the same flow, with pads for cout_lo and cin_hi.

3. **Top level.** The adder cell is placed twice, with the gating cell between the copies in the SIMD design. The placement is flattened, then the inter-block carries are drawn on metal 5, which no cell uses. Port labels go on the cells' pin stubs, and the power straps join through buses below the cells.

A first attempt placed all the gates of both adders in one set of rows. Its router ran out of tracks in the carry row, which led to the two-cell layout above.

The layout figures in this section are screenshots of MAGIC's own layout window.

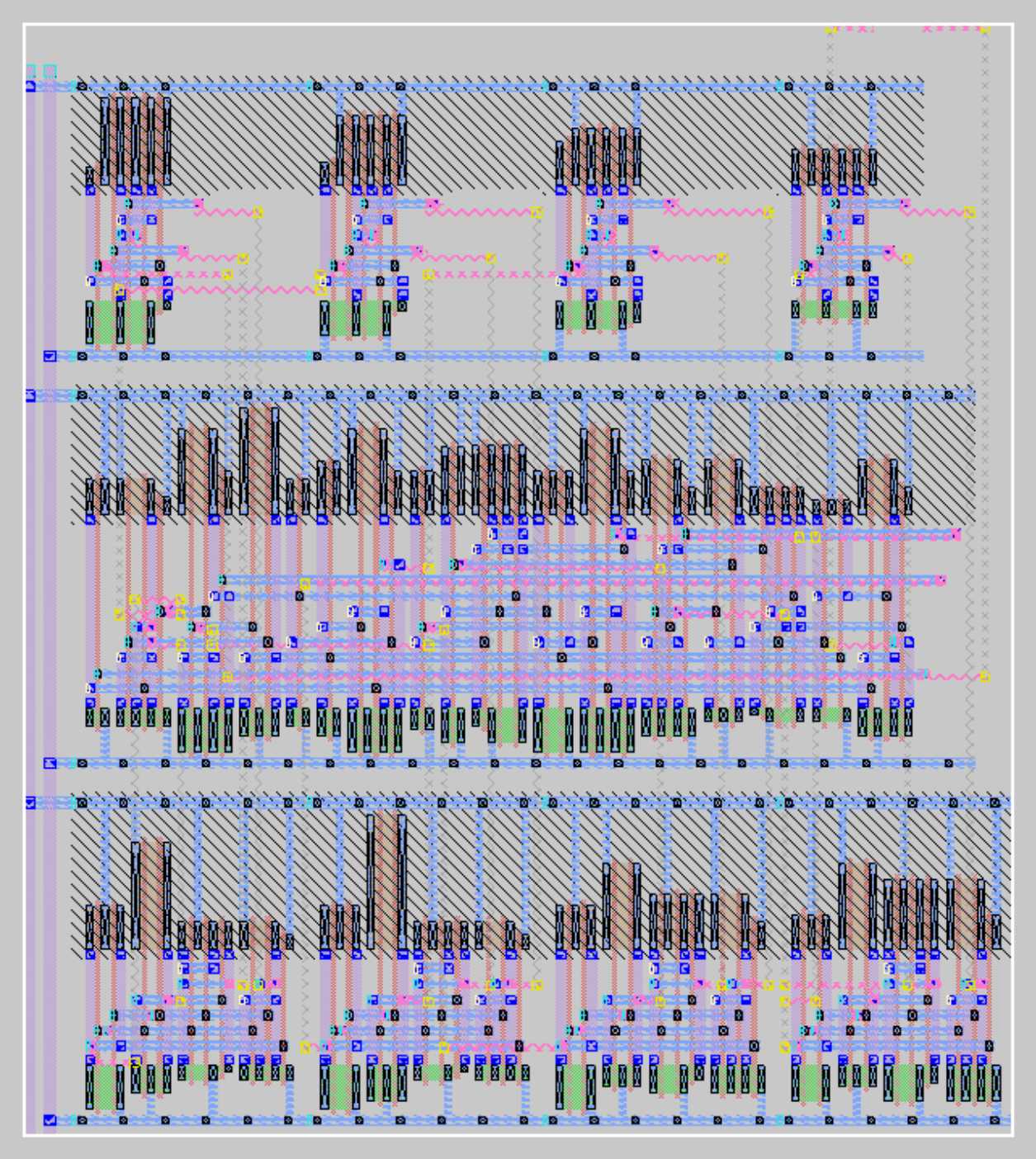


Figure 7: The Part I adder cell in MAGIC, with the two extra carry pads above the sum row.

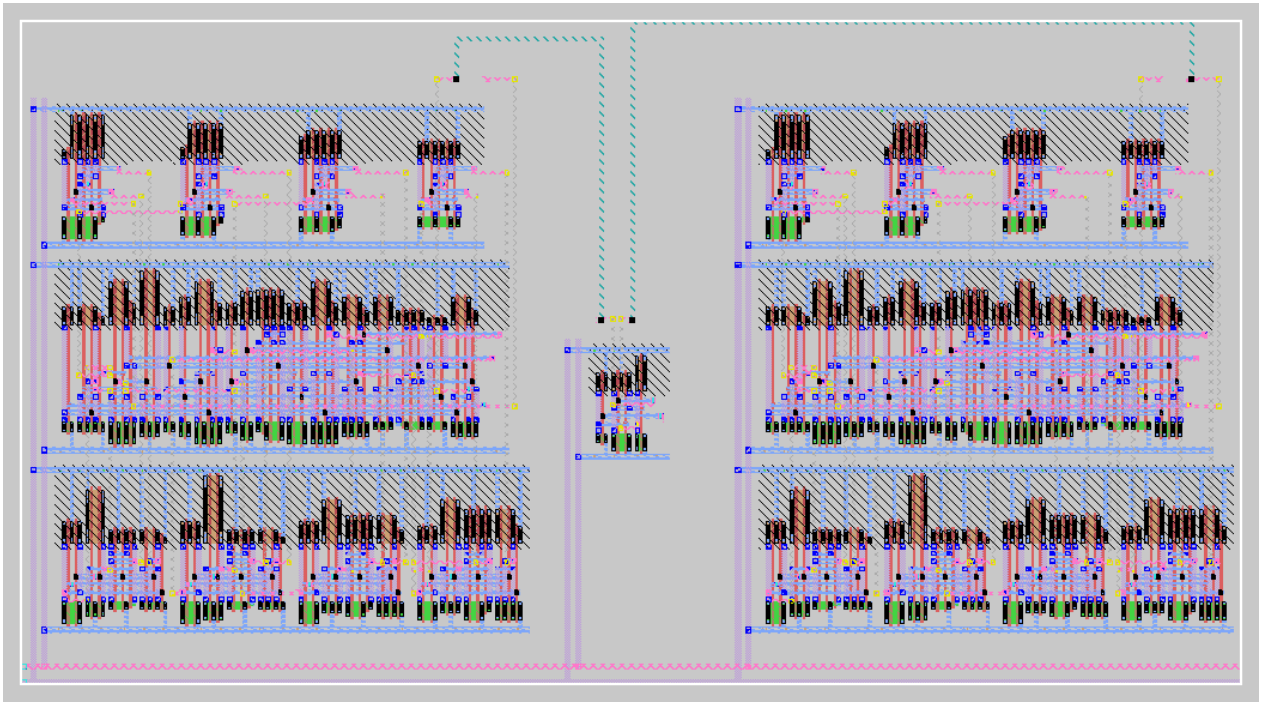


Figure 8: SIMD 8-bit adder in MAGIC: CLA0 on the left, the gating cell in the middle, CLA1 on the right. The dashed lines above are metal 5, carrying c4lo and cinh.

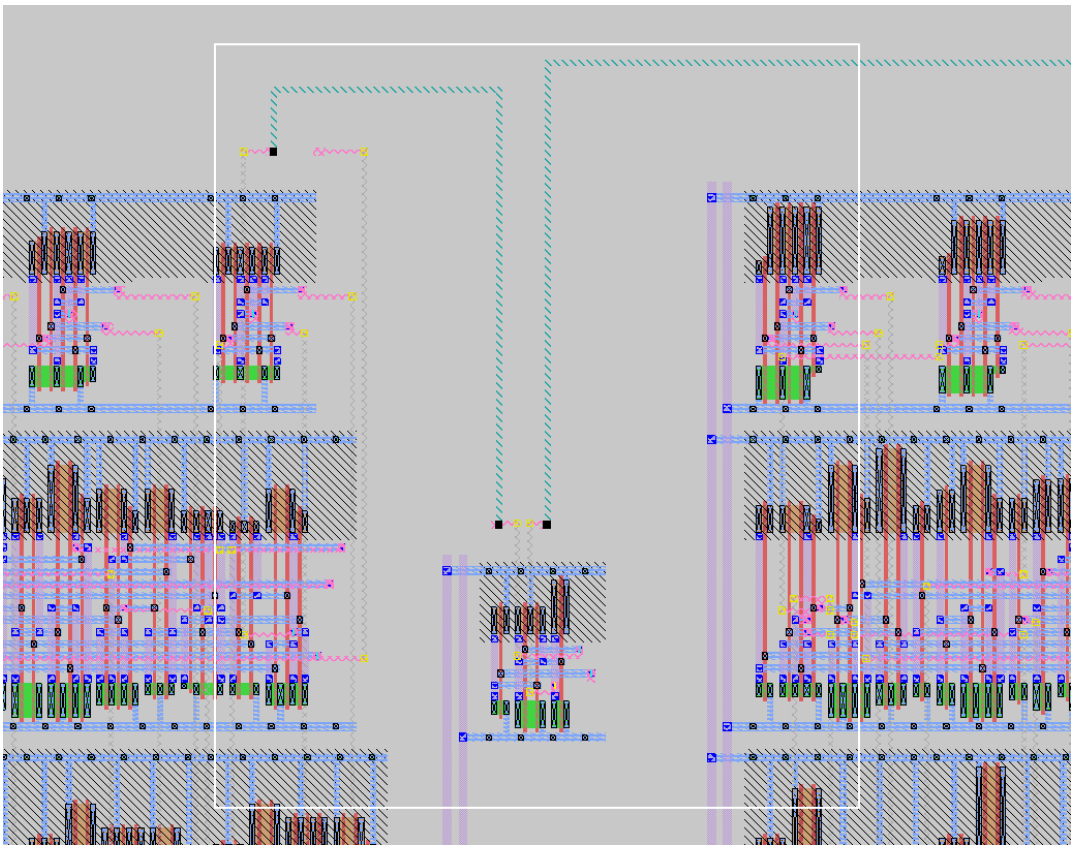


Figure 9: Close-up in MAGIC: the gating cell between CLA0 and CLA1 and its two metal 5 carries. The white rectangle is MAGIC's box, used to set the zoom.

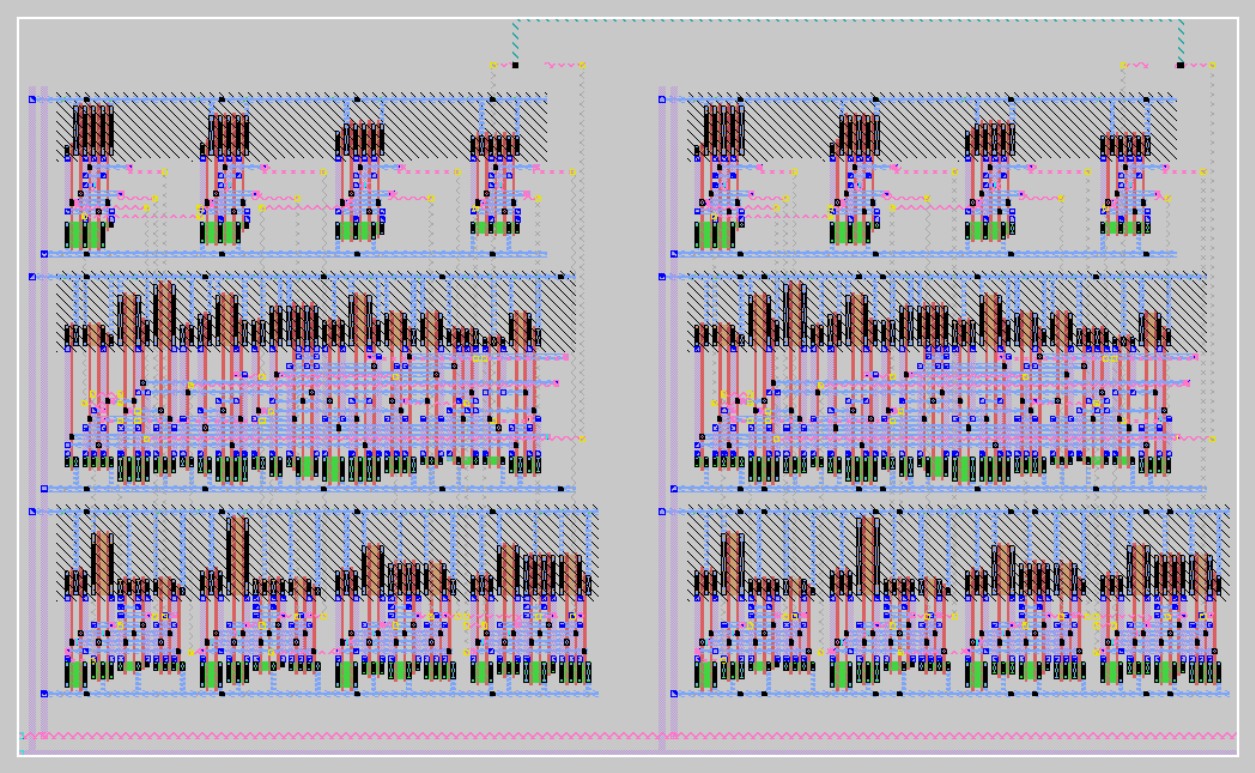


Figure 10: Baseline 8-bit adder in MAGIC: c4lo runs on metal 5 straight from CLA0 to CLA1.

Table 2: Layout checks and size.

Design	DRC	LVS	Transistors	Size (λ)	Area (μm^2)	Wire C (fF)
Baseline	0	match: 388 merged devices, 213 nets	388	1236 x 774	7749.0	766.5
SIMD	0	match: 396 merged devices, 218 nets	396	1413 x 774	8858.7	792.4

The SIMD layout is 14.3% larger. Most of that is the gating cell's slot between the two adders, not the 8 transistors themselves. LVS uses the project's own comparison script, as in Part I, because netgen was not available.

5 Worst-case propagation delay in both modes

An exhaustive search over all transitions of 17 or 18 inputs is too long, so the search is guided. Each case runs 13,312 single-input transitions in four Eulerian walks:

- every transition of CLA0's nine inputs, with CLA1 set to pass a carry through all four bits: A_{hi} = F, B_{hi} = 0, then A_{hi} = 0, B_{hi} = F;
- every transition of CLA1's eight inputs, with CLA0's carry-out held at 0 and then at 1.

These walks contain every path from a CLA0 input across the block boundary to the upper sums and C8. Delay runs from the adder pin to the last 50% crossing of the output, so false sums do not shorten it. Every output was also checked after each transition.

Table 3: Worst-case delay. tpd,pin is from the adder pin; tpd,drv includes the input driver.

Case	View	Transitions	Errors	tpd,pin (ps)	tpd,drv (ps)	Worst arc
Baseline	schematic	13312	0	855.22	934.52	a3 rise to s5 fall, A=F7 B=04 C0=0
Baseline	post-layout	13312	0	1030.86	1115.70	a3 rise to s5 fall, A=F7 B=04 C0=0
SIMD, mode 0	schematic	13312	0	920.09	991.01	a3 fall to s5 rise, A=FD B=07 C0=1 M=0
SIMD, mode 0	post-layout	13312	0	1109.77	1183.87	a3 fall to s5 rise, A=FF B=07 C0=1 M=0
SIMD, mode 1	schematic	13312	0	516.31	590.23	a5 rise to s7 rise, A=1F B=51 C0=0 M=1
SIMD, mode 1	post-layout	13312	0	570.22	649.82	a1 rise to s3 rise, A=F5 B=01 C0=0 M=1

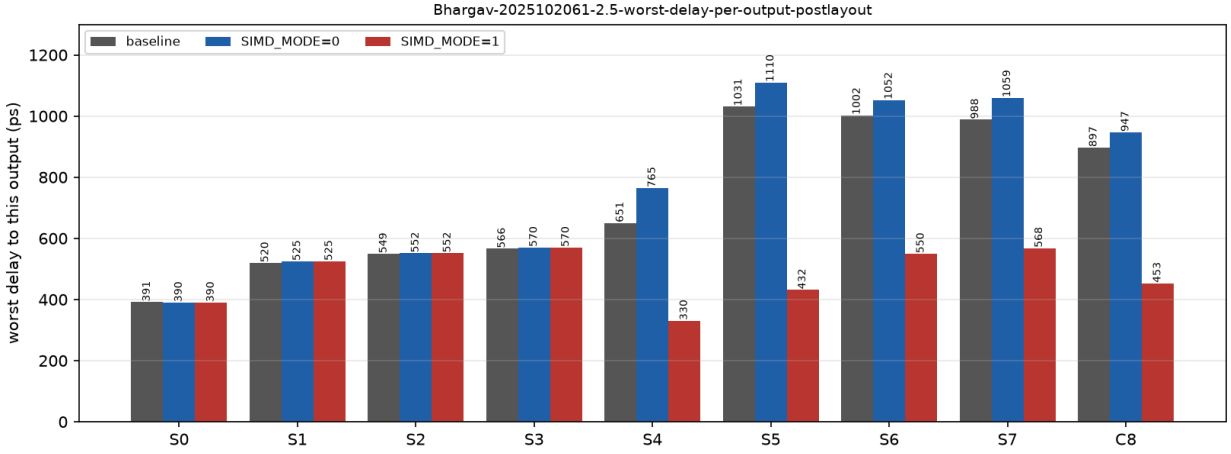


Figure 11: Worst post-layout delay to each output.

Reading the results.

- In 8-bit mode the worst path runs from a3, through CLA0's carry tree and the inter-block carry, into CLA1's sums. The gating cell adds 78.91 ps (7.65%) to tpd,max post-layout, and 64.87 ps (7.59%) on the schematic.
- In dual mode the carry never crosses, so the worst path stays inside one 4-bit block: 570.22 ps post-layout. This is close to Part I's 579.88 ps; the adder cell here was laid out again with the carry pads, and has the extra c4lo load.

6 Delay penalty of the gating cell

Worst-case carry propagation vector, A = 0xFF, B = 0x00, Cin_0 toggling. Delay from Cin_0 to S7, in mode 0 for the SIMD adder:

$$\text{Delay penalty } (\%) = \frac{t_{pd,SIMD} - t_{pd,baseline}}{t_{pd,baseline}} \times 100\%$$

Table 4: Cin to S7 on the worst-case vector. In mode 1, S7 and C8 do not change.

View	Case	Cin rise to S7	Cin fall to S7	Cin rise to C8	Cin rise to c4lo
schematic	Baseline	633.33	600.29	536.20	–
schematic	SIMD, mode 0	691.62	696.59	596.53	247.16
schematic	SIMD, mode 1	–	–	–	244.04
post-layout	Baseline	816.11	789.50	727.12	–
post-layout	SIMD, mode 0	877.83	903.93	781.59	323.84
post-layout	SIMD, mode 1	–	–	–	320.46

Table 5: Delay penalty, SIMD mode 0 against the baseline.

View	Cin rise to S7	Cin fall to S7
Schematic	+58.28 ps, 9.20%	+96.30 ps, 16.04%
Post-layout	+61.72 ps, 7.56%	+114.44 ps, 14.49%

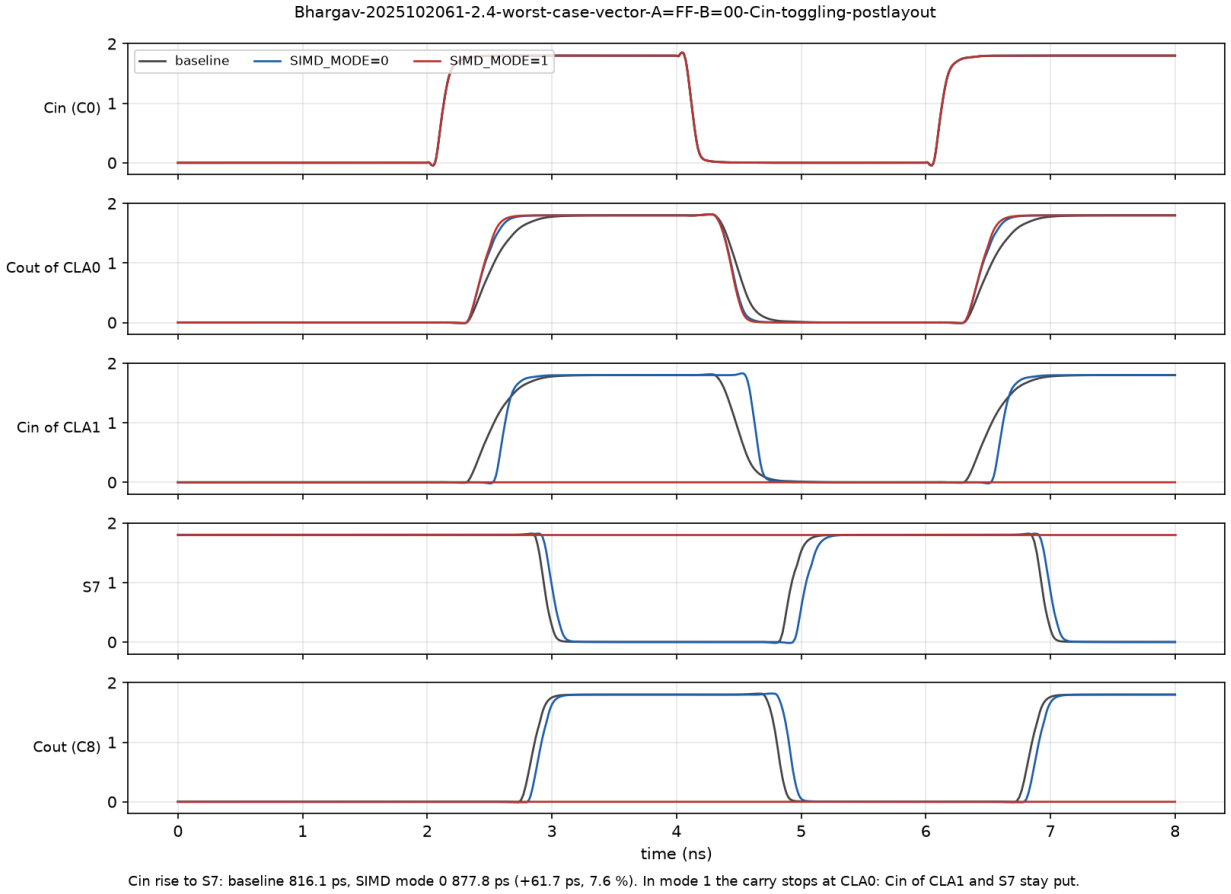


Figure 12: Post-layout waveforms on the worst-case vector, all three cases.

Why the penalty is smaller than the gate's own delay. The gate alone takes 110.46 ps, but the rising-edge penalty is only 61.72 ps. In the baseline, CLA0's carry-out drives CLA1's carry-in load directly. In the SIMD design it drives only the NAND2 input and the output load, so it switches sooner, which figure 12 shows. On the falling edge the penalty is larger, 114.44 ps.

7 Maximum frequency and power penalty

The maximum frequency uses the course formula, $f_{max} = 1/(t_{pd,max} + t_{setup} + t_{hold})$, with the Part I flip-flop's post-layout $t_{setup} = 38.0$ ps and $t_{hold} = 45.7$ ps.

Table 6: Course-formula maximum frequency, post-layout.

Case	tpd,max (ps)	Period (ps)	fmax (GHz)
Baseline	1030.86	1114.56	0.897
SIMD, mode 0	1109.77	1193.47	0.838
SIMD, mode 1	570.22	653.92	1.529

The formula's clock is too short for some inputs. At each design's own course-formula

clock, random vectors gave wrong outputs at the end of some cycles: 10 of 200 for the baseline and 46 for SIMD mode 0. These vectors change several inputs at once, which is slower than the single-input worst case the search found, as in Part I. Power measured while outputs fail is not trustworthy. So the shortest clock at which all three cases settle on the same 200 random vectors was found:

Table 7: Wrong output samples on 200 random vectors, post-layout.

Period (ps)	**Baseline**	**SIMD, mode 0**	**SIMD, mode 1**
1250	6	17	0
1300	2	5	0
1350	0	2	0
1400	0	0	0
1500	0	0	0

All three cases settle at 1400 ps, so power is compared there. All three get identical input vectors: 10 cycles as the task asks, and 200 vectors as a check. Power is the adder's own supply only: drivers and load inverters use a separate source.

$$\text{Power penalty (\%)} = \frac{P_{SIMD,\,mode\,0} - P_{baseline}}{P_{baseline}} \times 100\%$$

Table 8: Average power at 1400 ps, post-layout. 0 wrong outputs in every run.

Case	**10 cycles (μW)**	**200 vectors (μW)**
Baseline	1695.2	1876.3
SIMD, mode 0	1720.7	1919.5
SIMD, mode 1	1592.4	1713.7

Power penalty in 8-bit mode: 1.50% over 10 cycles, and 2.30% over 200 vectors. The gating cell adds little switching: 8 small transistors, with the mode inverter idle. Dual mode uses the least power, because a carry stopped at the boundary does not ripple through CLA1.

8 Energy-delay product

$$EDP = P_{avg} \times t_{pd}^2$$

Table 9: Energy-delay product, with the 10-cycle power at the common clock and the post-layout tpd,max.

Case	**Pavg (μW)**	**tpd,max (ps)**	**EDP (J s)**
Baseline	1695.2	1030.86	1.801e-21
SIMD, mode 0	1720.7	1109.77	2.119e-21
SIMD, mode 1	1592.4	570.22	5.178e-22

In 8-bit mode the SIMD adder's EDP is 17.6% higher than the baseline's. Delay counts twice in EDP, so the 7.7% delay penalty dominates the 1.5% power penalty. In dual mode the EDP is far lower, because each add is only 4 bits long.

9 Summary and limits

Table 10: Cost of reconfigurability: SIMD mode 0 against the baseline, post-layout.

Measure	Baseline	SIMD mode 0	Penalty
Cin rise to S7 (ps)	816.11	877.83	7.56%
tpd,max (ps)	1030.86	1109.77	7.65%
Power, 10 cycles (μW)	1695.2	1720.7	1.50%
EDP (J s)	1.801e-21	2.119e-21	17.6%
Transistors	388	396	+8
Area (μm^2)	7749.0	8858.7	14.3%

- **Guided search.** The worst-case search is guided and single-input; it is not exhaustive. Multi-input changes are slower, as the settle test shows, and their exact worst case was not searched.
- **Power clock.** Power is compared at a common verified clock, not at each design's course-formula fmax, because at that clock some vectors fail.
- **Area.** It is the bounding box, including gaps and routing channels.
- **Wire resistance.** Not extracted, as in Part I: MAGIC's resistance extraction crashed.

A Netlists

Generated from `part2/cells/simd.py`. The full files, with the Part I cells, are in `part2/schematics/`.

Carry-gating cell

```
.subckt carry_gate_n1_i2 cout_lo mode cin_hi vdd gnd
Xinvm mode modeb vdd gnd inv_x1
Xnandg cout_lo modeb gn vdd gnd nand2_x1
Xinvg gn cin_hi vdd gnd inv_x2
.ends carry_gate_n1_i2
```

SIMD 8-bit adder, top level

```
.subckt simd8 a0 a1 a2 a3 a4 a5 a6 a7 b0 b1 b2 b3 b4 b5 b6 b7 c0 mode s0 s1 s2 s3 s4 s5 s6
+ s7 c8 c4lo vdd gnd
Xcla0 a0 a1 a2 a3 b0 b1 b2 b3 c0 s0 s1 s2 s3 c4lo vdd gnd cla4_final
Xgate c4lo mode cinh vdd gnd carry_gate_n1_i2
Xcla1 a4 a5 a6 a7 b4 b5 b6 b7 cinh s4 s5 s6 s7 c8 vdd gnd cla4_final
.ends simd8
```

Baseline 8-bit adder, top level

```
.subckt base8 a0 a1 a2 a3 a4 a5 a6 a7 b0 b1 b2 b3 b4 b5 b6 b7 c0 s0 s1 s2 s3 s4 s5 s6 s7 c8
+ vdd gnd
Xcla0 a0 a1 a2 a3 b0 b1 b2 b3 c0 s0 s1 s2 s3 c4lo vdd gnd cla4_final
Xcla1 a4 a5 a6 a7 b4 b5 b6 b7 c4lo s4 s5 s6 s7 c8 vdd gnd cla4_final
.ends base8
```

B Reproducing the results

From `part2/cells/`, in order: `simd.py gate`, `netlists`, `cells`, `layout_h simd base`, `search` for each case and view, `penalty`, `settle`, `power 1400`, `waves`, `summary`, then `make_report.py`.